DSA0502 QUERY PROCESSING FOR DATASCIENCE

CO2 – AT2 – SCHEMA ANALYSIS AND VIVA VOCE

NAME : DineshK

REG NO : 192424030

1. UNIVERSITY COURSE REGISTRATION SYSTEM

A university database contains the following tables:

- Student(StudentID, Name, DepartmentID)
- Department(DepartmentID, DepartmentName)
- Course(CourseID, CourseName, FacultyID)
- Faculty(FacultyID, FacultyName)
- Enrollment(StudentID, CourseID, Semester)

A student can enroll in multiple courses, and each course can have multiple students.

VIVA QUESTIONS

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

SOLUTION:

SCHEMA ANALYSIS

Tables

Student

Column	Description
StudentID	Unique student ID (Primary Key)
Name	Student name
DepartmentID	Department of the student (Foreign Key)

DEPARTMENT

Column	Description
DepartmentID	Primary Key
DepartmentName	Department name

COURSE

Column	Description
CourseID	Primary Key
CourseName	Name of the course
FacultyID	Faculty teaching the course

FACULTY

Column	Description
FacultyID	Primary Key
FacultyName	Faculty name

ENROLLMENT

Column	Description
StudentID	Foreign Key
CourseID	Foreign Key
Semester	Semester enrolled

RELATIONSHIPS

- One Department → Many Students
- One Faculty → Many Courses
- One Student → Many Courses
- One Course → Many Students

The **Enrollment** table creates a **Many-to-Many relationship** between Student and Course.

VIVA VOCE

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

Answer

A student can enroll in multiple courses, and each course can have many students. This is a many-to-many relationship. The Enrollment table stores these relationships without duplicating data and keeps the database normalized.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

Answer

Add a **Grade** column to the Enrollment table.

Example:

Enrollment(StudentID, CourseID, Semester, Grade)

This allows storing a separate grade for every student in every course.

CODE:

```
students = [
    {"StudentID": 1, "Name": "Dinesh"},
    {"StudentID": 2, "Name": "Rahul"}
]
courses = [
    {"CourseID": 101, "CourseName": "Python"},
    {"CourseID": 102, "CourseName": "Database"}
]
enrollment = [
    {"StudentID": 1, "CourseID": 101, "Semester": "III"},
    {"StudentID": 1, "CourseID": 102, "Semester": "III"},
    {"StudentID": 2, "CourseID": 101, "Semester": "III"}
]
print("Enrollment Details")
for e in enrollment:
    student = next(s["Name"] for s in students if s["StudentID"] == e["StudentID"])
    course = next(c["CourseName"] for c in courses if c["CourseID"] == e["CourseID"])
    print(student, "-", course, "-", e["Semester"])
```

OUTPUT:

Enrollment Details

Dinesh - Python - III

Dinesh - Database - III

Rahul - Python – III

2. HOTEL RESERVATION SYSTEM

Tables:

- Customer(CustomerID, Name)
- Room(RoomID, RoomType)
- Reservation(ReservationID, CustomerID, RoomID, CheckIn, CheckOut)

Rooms can be booked by different customers at different times.

Viva Questions

Q1. Why should reservation details be maintained separately from room details?

Q2. How would you prevent double booking of rooms using the schema?

SOLUTION:

SCHEMA ANALYSIS

Tables

CUSTOMER

Column	Description
CustomerID	Primary Key
Name	Customer name

ROOM

Column	Description
RoomID	Primary Key
RoomType	Single, Double, Deluxe

RESERVATION

Column	Description
ReservationID	Primary Key
CustomerID	Foreign Key
RoomID	Foreign Key

CheckIn	Check-in date
CheckOut	Check-out date

RELATIONSHIPS

- One Customer → Many Reservations
- One Room → Many Reservations (at different dates)

Reservation stores booking history.

VIVA VOCE

Q1. Why should reservation details be maintained separately from room details?

Answer

Room details are permanent (Room ID, Room Type), while reservation details change for every booking (customer, check-in, check-out). Keeping them separate avoids data duplication and stores booking history efficiently.

Q2. How would you prevent double booking of rooms using the schema?

Answer

Before inserting a new reservation, check whether another reservation already exists for the same room with overlapping check-in and check-out dates. If an overlap exists, reject the booking.

CODE:

```
reservations = [
    {"RoomID": 101, "CheckIn": 1, "CheckOut": 5}
]
new_checkin = 3
new_checkout = 6
room = 101
booked = False
for r in reservations:
    if r["RoomID"] == room:
```

```python
        if not (new_checkout <= r["CheckIn"] or new_checkin >= r["CheckOut"]):
            booked = True
if booked:
    print("Room already booked")
else:
    print("Room available")
```

OUTPUT

Room already booked